

Algoritmi e Strutture Dati – 2/7/2024 – Parte A

A1 – Complessità – Punti ≥ 8

Trovare un limite superiore ed un limite inferiore, i più stretti possibili, per la seguente equazione di ricorrenza, utilizzando il *metodo di sostituzione*.

$$T(n) = \begin{cases} 1 & n < 4 \\ 7T(\lfloor n/4 \rfloor) + n\sqrt{n} & n \geq 4 \end{cases}$$

A2 – Più o meno – Punti ≥ 10

Scrivere un algoritmo

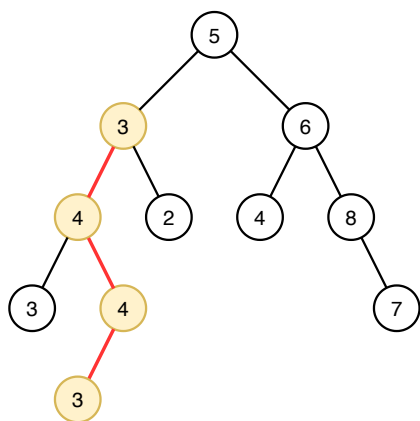
int longestPlusMinus(TREE *T*, **int** *k*)

che prenda in input un albero binario contenente numeri interi (accessibili tramite l'attributo *key* dei nodi) e restituisca la lunghezza del cammino nodo-foglia *più-o-meno-k* più lungo contenuto nell'albero.

Un cammino nodo-foglia parte da un qualunque nodo *t* e raggiunge uno qualsiasi dei nodi foglia contenuti nel sottoalbero radicato in *t*. Tale cammino è detto *più-o-meno-k* se il valore contenuto in ogni nodo del cammino differisce dal valore contenuto nel padre di tale nodo per al più *k*.

Discutere informalmente la correttezza della soluzione proposta e calcolare la complessità computazionale.

Nell'albero seguente e con $k = 1$, il più lungo cammino nodo-foglia evidenziato è quello contenuto nell'albero e l'algoritmo deve restituire 3.



A3 – Circuiti matriciali – Punti ≥ 12

State sviluppando un generatore casuale di mappe per un videogioco di corse automobilistiche. Ogni mappa è rappresentata da una matrice booleana *M* di dimensione $n \times n$, dove le celle con valore **true** rappresentano ostacoli e non possono essere attraversate, mentre le celle con valore **false** sono libere e attraversabili. Ogni auto può muoversi in quattro direzioni: sinistra, destra, alto e basso. Le auto non possono muoversi in diagonale o in altre direzioni non ortogonali.

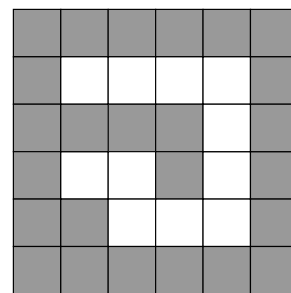
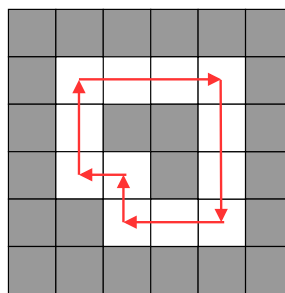
Il vostro compito è scrivere un algoritmo

boolean hasCircuit(**boolean**[][] *M*, **int** *n*)

che prenda in input la matrice e verifichi se all'interno di essa esiste un circuito, ovvero un percorso per cui sia possibile partire da una cella, muoversi tra le celle in una delle quattro direzioni consentite, e eventualmente tornare alla cella di partenza senza attraversare celle già percorse.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, nella mappa di esempio a sinistra, esiste un circuito; nella mappa di destra, no.



Algoritmi e Strutture Dati – 2/7/2024 – Parte B

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

B1 – Vie di fuga – Punti ≥ 8

Utilizzando un sofisticato algoritmo, un famoso professore ha disposto gli studenti in aula per uno scritto; secondo le regole d'Ateneo deve ora verificare che la disposizione sia "sicura", cioè che tutti gli studenti abbiano a disposizione vie di fuga disgiunte.

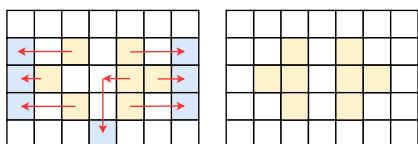
Il problema è descritto come segue:

- l'aula è data da una griglia $R \times C$, dove (r, c) rappresenta la posizione in riga r e colonna c ;
- ci sono N studenti, ciascuno descritto dalle coordinate (r_i, c_i) , cioè la sua posizione in aula;
- per fuggire, ogni studente può muoversi lungo la riga di sedili (a sinistra o a destra) o scavalcare la riga di sedili (in avanti o indietro); non può muoversi in diagonale perché gli informatici non sono molto atletici;
- tutte le posizioni ai bordi sono considerate potenziali punti di arrivo per la fuga;
- una via di fuga è un percorso dalla posizione dello studente al bordo, effettuato tramite le mosse legali descritte sopra;
- due vie di fuga sono disgiunte se non hanno nessuna posizione in comune.

Descrivere un algoritmo che, dato in input le dimensioni R, C e le coordinate degli N studenti, restituisca **true** se e solo se esistono N vie di fuga disgiunte.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Qui sotto, l'aula è rappresentata dalla griglia, le celle in oca sono gli studenti e le celle in azzurro le posizioni di arrivo delle vie di fuga evidenziate in rosso. Nella griglia a sinistra, è possibile trovare vie di fuga disgiunte per tutti; nella griglia a destra, no.



B2 – Hamilton – Punti ≥ 10

Scrivere un algoritmo

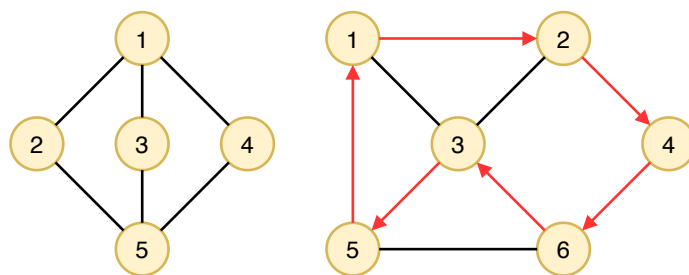
boolean hamilton(GRAPH G)

che prenda in input un grafo non orientato G e restituisca **true** se il grafo contiene un ciclo hamiltoniano, **false** altrimenti.

Un *ciclo hamiltoniano* in un grafo è un percorso chiuso che attraversa tutti i nodi del grafo esattamente una volta, tornando infine al nodo di partenza. Come punto di partenza, potete utilizzare il nodo 1.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Nella figura seguente, il grafo di sinistra non contiene un ciclo hamiltoniano, mentre il grafo di destra contiene il ciclo evidenziato dalle frecce rosse.



B3 – Norma L1 – Punti ≥ 12

La *norma L1* fra due vettori V_1 e V_2 di uguale dimensione n è definita come segue:

$$\|V_1 - V_2\|_1 = \sum_{i=1}^n |V_1[i] - V_2[i]|$$

Scrivere un algoritmo

int maxL1(**int**[] V_1 , **int**[] V_2 , **int** n_1 , **int** n_2)

che prenda in input due vettori V_1 e V_2 di interi di dimensione n_1 e n_2 , e restituisca la massima norma L1 ottenibile selezionando due sottosequenze di uguale dimensione prese dai vettori originali.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, si consideri $V_1 = [1, 2, 3]$ e $V_2 = [1, 5, 2]$.

- Prendendo come sottosequenze i vettori tutti interi, si ottiene: $\| [1, 2, 3] - [1, 5, 2] \|_1 = |1 - 1| + |2 - 5| + |2 - 3| = 4$
- Prendendo come sottosequenze i vettori $[1, 3]$ e $[5, 2]$, si ottiene: $\| [1, 3] - [5, 2] \|_1 = |1 - 5| + |2 - 3| = 5$

Le sottosequenze $[1, 3]$ e $[5, 2]$ sono in effetti quelle che danno origine alla norma L1 e l'algoritmo dovrà restituire 5.